

From our benchmark to an external Transformer

Felix Borck | Proposed test in DeepMind's Tracr repository

PLANNED, NOT RUN • Upstream interfaces checked; intervention adapter still needed.

Why Tracr?

Tracr compiles programs into Transformer weights. Its reversal program provides a known computation for independent checking. This is a third-party compiled Transformer, not a pretrained language model.

The rivals: an address, or a copied answer?

Address: retrieve the recipient token at the transferred position. **Answer copy:** reproduce the donor's answer. Transfer donor query 1's intermediate address 2 into recipient query 0; count positions from zero, excluding BOS.

Stage	Recipient	Donor	Address predicts	Copy predicts
1 Shared prediction	[A,B,C,D]	[W,X,C,Z]	C	C
2 Separating condition	[A,B,C,D]	[W,X,Y,Z]	C	Y
3 Prediction check	[A,B,U,D]	[W,X,Y,Z]	U	Y

These are predictions, not observations. The unpatched recipient answers D in every row. The decisive change preserves the donor address while changing its answer.

Build only what this test needs

- 1. Qualify the task and patch.** Compile reversal once; reproduce native outputs. Add a forward adapter, check the actual copied coordinates, BOS offset, identities and numerical precision.
- 2. Freeze the narrowing rule.** Declare rivals, allowed cells, budget, stopping rules and sequential uncertainty. Keep compiler answer labels out of the decision process.
- 3. Log the trajectory.** Show both candidates surviving the shared condition; use their disagreement to select the next condition. Preserve both, neither and invalid outcomes.
- 4. Test fresh predictions.** Use untouched sequence families. A later test transfers to another program and compares selection policies at equal budgets.

What a successful demonstration would establish: an explicit intervention distinguished two declared interpretations, and the survivor predicted new outcomes. It would not by itself prove adaptive superiority or identify a native LLM mechanism.